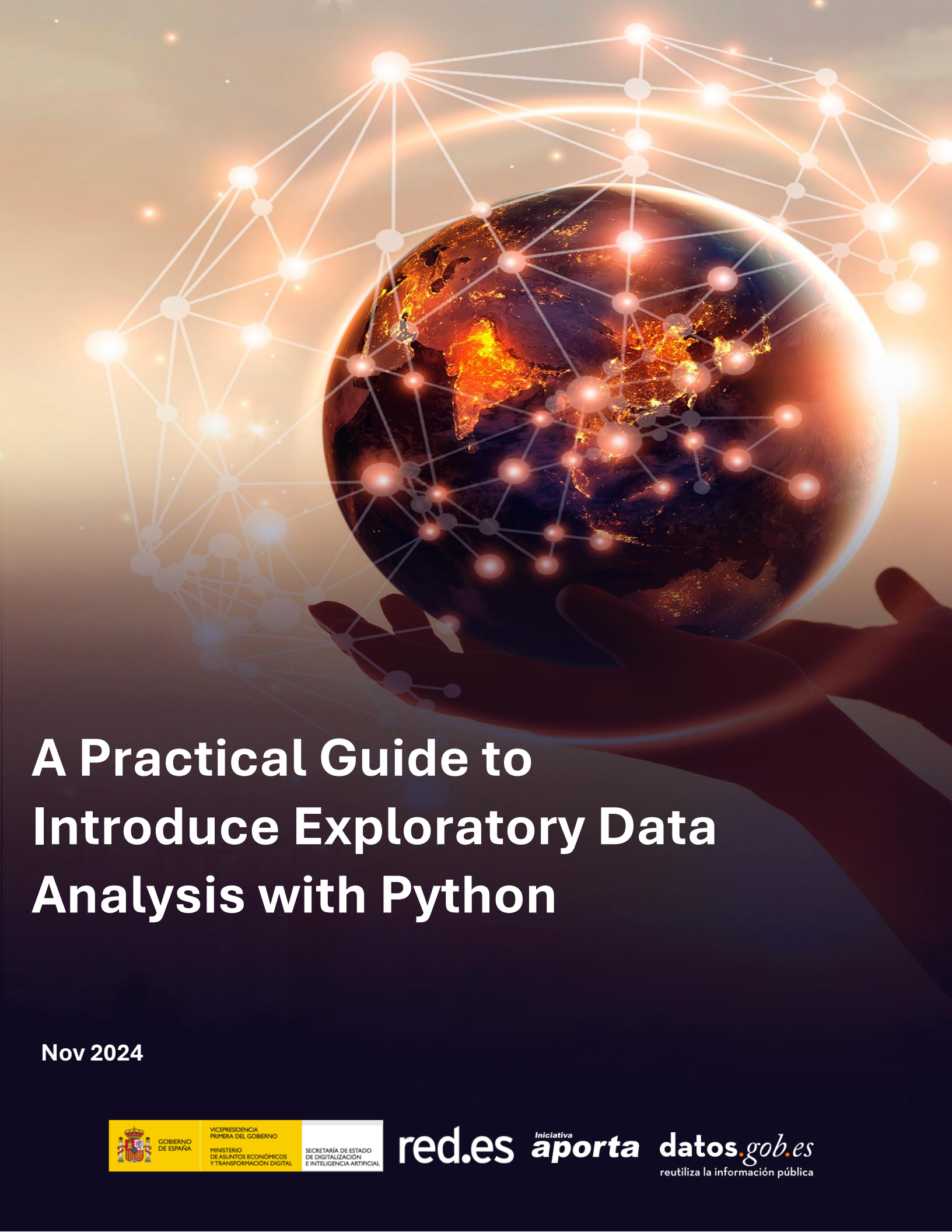


A hand holding a globe with a network overlay. The globe shows the Earth with glowing orange and yellow lights representing data or activity. A network of white lines and dots connects various points across the globe and extends into the background, suggesting a global network or data flow.

A Practical Guide to Introduce Exploratory Data Analysis with Python

Nov 2024



VICEPRESIDENCIA
PRIMERA DEL GOBIERNO
MINISTERIO
DE ASUNTOS ECONÓMICOS
Y TRANSFORMACIÓN DIGITAL

SECRETARÍA DE ESTADO
DE DIGITALIZACIÓN
E INTELIGENCIA ARTIFICIAL

red.es

Iniciativa

aporta

datos.gob.es
reutiliza la información pública

This document has been prepared within the framework of the Aporta Initiative (datos.gob.es), developed by the Ministry for Digital Transformation and Public Administration through the Public Business Entity Red.es, in collaboration with the Data General Directorate.

Legal Notice: This work is licensed under a Creative Commons Attribution 4.0 International License (CC BY 4.0). It is permitted to reproduce, distribute, publicly communicate, and transform to create a derivative work, without any restriction, as long as the rights holder is cited (Ministry of Economic Affairs and Digital Transformation through the Public Business Entity Red.es). To view a copy of this license, visit: <https://creativecommons.org/licenses/by/4.0>

TABLE OF CONTENTS

INTRODUCTION.....	1
1. METHODOLOGY.....	2
2. EXPLORATORY DATA ANALYSIS	2
2.1. DESCRIPTIVE ANALYSIS.....	3
2.1.1. EXPERIMENT	4
2.2. ADJUSTMENT OF VARIABLE TYPES	8
2.2.1. EXPERIMENT	8
2.3. DETECTION AND TREATMENT OF MISSING DATA	10
2.3.1. DETECTION OF MISSING DATA	10
2.3.2. TREATMENT OF MISSING DATA	12
2.4. DETECTION AND TREATMENT OF OUTLIERS	14
2.4.1. OUTLIERS DETECTION.....	15
2.4.2. OUTLIER REMOVAL	19
2.5. CORRELATION ANALYSIS BETWEEN VARIABLES	20
2.5.1. EXPERIMENT	21
3. AUTOMATED EXPLORATORY DATA ANALYSIS.....	23
3.1. EXPERIMENT.....	23
4. CONCLUSIONS.....	25
5. NEXT STEPS	26

LIST OF ILLUSTRATIONS

Figure 1 - Representation of the set of stages of exploratory data analysis **Error! Bookmark not defined.**

Figure 2 – EDA First Stage – Descriptive Analysis	3
Figure 3 - General information about the dataset.....	5
Figure 4 – Numerical variables histograms	7
Figure 5 – EDA Second Stage - Adjustment of variable types	8
Figure 6 - Unique categories of the transformed categorical variables	9
Figure 7 -EDA Third Stage - Detection and Treatment of Missing Data	10
Figure 8 - Results of the Missing Values Analysis	11
Figure 9 - Results of the Missing Data Treatment	13
Figure 10 -EDA Fourth Stage -Outliers Identification	14
Figure 11 - Variable ' O3 ($\mu\text{g}/\text{m}^3$) Histogram	15
Figure 12 - Statistics on the distribution of the variable 'O3 ($\mu\text{g}/\text{m}^3$)'	17
Figure 13 - Box plot of the variable ' $\mu\text{g}/\text{m}^3$ '	17
Figure 14 - Bar chart of the categorical variable 'Provincia'	18
Figure 15 - Box plot of the variable 'O3 ($\mu\text{g}/\text{m}^3$)' before and after outlier removal	19
Figure 16 - Unique categories of the variable 'Provincia' after outlier removal	20
Figure 17 – EDA Fifth Stage – Variables Correlation	20
Figure 18 - Correlation matrix of numerical variables	22
Figure 19 - General summary of the dataset generated by 'ydata-profiling'	24
Figure 20 - Automatically generated information about the variable 'Provincia'	25

INTRODUCTION

This document constitutes an adaptation to the Python programming language of [A Practical Guide to Exploratory Data Analysis with R \(Introduction\) published by the Aporta Initiative in 2021](#). The dataset and the basic structure of the referenced analysis are maintained with the intention of facilitating the comparison between both programming languages, allowing users to identify syntactic and implementation differences, and providing a valuable resource for those working in environments where both languages are used. Additionally, this approach allows focusing on the particularities of each language without the differences in data or the structure of the analysis adding unnecessary complexity to the learning process. As an added value, this new version incorporates sections dedicated to emerging trends in the field, such as automated exploratory analysis and data profiling tools, thus responding to the current needs of the sector in terms of efficiency and scalability.

Exploratory Data Analysis (EDA) represents a **critical step prior to any statistical analysis**. Its relevance derives both from the need to thoroughly understand the data before analyzing it and from verifying the fulfillment of statistical requirements that will ensure the validity of subsequent analyses.

The statistical techniques that make up **EDA allow for unraveling the intrinsic nature of the data, characterizing its main attributes, and discovering the interrelationships between variables**. This systematic process lays the foundation for a deep understanding of the dataset and underpins the robustness of subsequent analyses. The initial exploration reveals crucial aspects such as possible data entry errors, patterns of missing values, significant correlations between variables, and informational redundancies that could affect the quality of the analysis.”

Paradoxically, despite its fundamental role in ensuring consistent and accurate results, the exploratory phase is often minimized in data reusing processes. This guide addresses this issue by presenting both **traditional methodologies and modern approaches to EDA**, including automated tools that facilitate the systematic exploration of large datasets.

The debate on the precise delineation of the processes that make up exploratory analysis remains current in the scientific community. While some experts consider data cleaning as an independent preliminary phase, the intricate interrelation between exploration and cleaning, along with their dependence on the specific context of the data, suggests the convenience of an integrated approach. This introductory guide details a series of tasks that constitute the minimum set to be addressed **to ensure an acceptable starting point for effective data reuse**.

This guide is aimed both at those who are new to data analysis and those who seek to systematize the processes of EDA. Using the Python¹ programming language, key concepts are illustrated, and concrete examples are provided to facilitate the understanding and application of the techniques learned.

¹ To achieve the maximum understanding of the scope of this guide, it is recommended to have basic competencies in the Python language ([Aporta Initiative resources and exercises in Python](#)), which is chosen to illustrate, through examples, the different stages involved in EDA. If not, we still encourage you to continue reading this guide as it includes an interesting bibliography that, in addition to helping you understand EDA, will allow you to learn and make the most of this powerful programming language.

1. METHODOLOGY

This guide takes a practical approach based on experiential learning, allowing users to **become familiar with analysis techniques using real public data and open-source technological tools** at no cost. All developed materials, including data and source code, are made publicly available to facilitate both the replication of the analysis and its adaptation to other study contexts.

As a tool for the **practical case**, the Python programming language and the Jupyter Notebook development environment in Google Colab have been used. You can follow the guide by running the cells of the notebook published in the [official repository](#), ensuring the reproducibility of the results.

The selection of Python as the programming language is due to its prominent position in the field of data analysis, where it stands out for **combining an intuitive syntax with advanced analytical capabilities**. Although the guide includes the necessary code snippets to perform each task, the emphasis is placed on the conceptual understanding of the processes and the explanation of the key functionalities that Python offers for exploratory analysis. This approach prioritizes expository and didactic clarity over code optimization, thus facilitating understanding for different levels of technical experience.

Maintaining consistency with the previous guide, the same dataset has been chosen, specifically, [the Castilla and León Air Quality Records](#), available in the [datos.gob.es](#) portal. As already explained in the version implemented in R, the suitability of this dataset lies both in its social relevance and its technical characteristics, making it an ideal case study to illustrate different exploratory analysis techniques.

2. EXPLORATORY DATA ANALYSIS

To create this guide, we have taken as a reference the exploratory data analysis described in the book "Python Data Science Handbook" by Jake VanderPlas (Second Edition, 2023) "available for free on its [first edition](#)", including a large number of practical examples. The EDA we propose will follow these steps:

1. **Perform a descriptive analysis of the variables** to obtain a representative idea of the dataset.
2. **Adjust the variable types** to ensure consistency for subsequent operations.
3. **Detect and handle missing data** to properly process numerical variables. Missing data are values not recorded in some observations, and it is essential to manage them correctly to avoid biases and issues in the analysis.
4. **Identify and treat outliers** to prevent them from distorting future statistical analyses.
5. **Conduct a numerical and graphical examination of the relationships between the analyzed variables** to determine the degree of correlation between them, allowing the prediction of one variable's behavior based on the others.

The following chart (Figure 1) schematically represents the set of stages of exploratory data analysis described in the contents of this guide.

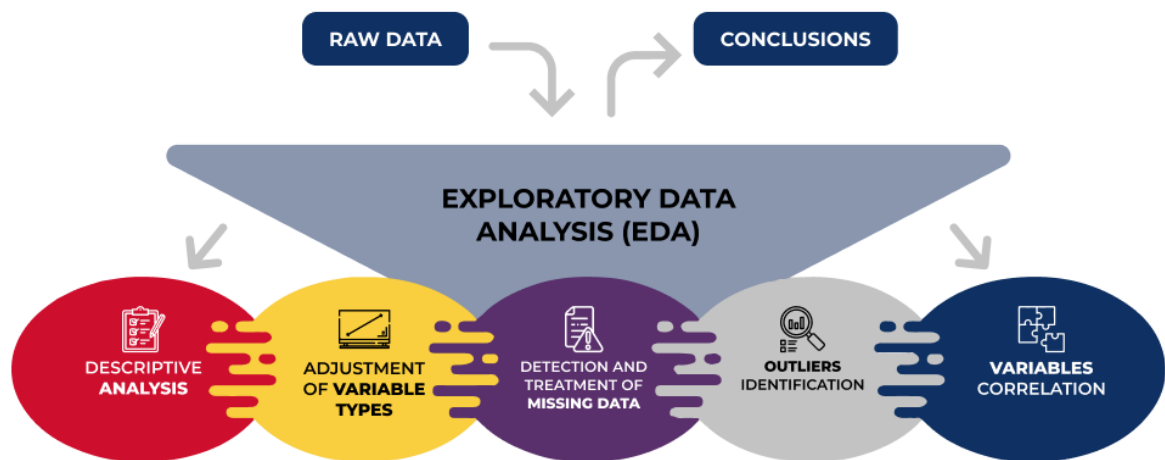


Figure 1 - Representation of the set of stages of exploratory data analysis

Let's take a detailed look at each of the proposed stages for conducting exploratory data analysis. Each chapter includes the section 'Experiment,' which, through the practical application of various functions in Python, will help you understand the concepts explained.

2.1. DESCRIPTIVE ANALYSIS



Figure 1 – EDA First Stage – Descriptive Analysis

Once the dataset on air quality records in Castilla and León has been obtained from the open data catalog (it can be downloaded directly by accessing [this link](#)) we will proceed to **perform an initial characterization of the dataset** to understand its structure and content. To do this, we will combine two complementary approaches: on one hand, the application of descriptive statistics techniques that will provide us with a quantitative view of the variables and their characteristics; on the other hand, the generation of visualizations that will help us intuitively understand the distribution patterns present in the data. This initial recognition phase lays the foundation for any deeper and more targeted analysis.

2.1.1. EXPERIMENT

For our analysis, we will use the aforementioned dataset, which we will assign to the object **calidad_aire** in our code. This dataset will serve as the basis for implementing and demonstrating all the exploratory analysis techniques presented in the guide.

The initial exploration will be carried out using various Python functions specifically designed to provide a **global perspective of the structure and content of the data**. These same functions will be used repeatedly throughout the analysis to monitor how different transformations and processes affect the characteristics of the dataset.

For the initial acquisition of the dataset, automation has been prioritized using the **read_csv()** function, which retrieves the information directly from the open data catalog. While this method may slightly increase the initial execution time, it ensures the reproducibility of the process and simplifies the workflow. Alternatively, if greater control over the process is preferred or faster execution is needed, there is the option to download the file from the provided link beforehand and load it from a local directory.

```
# Cargar las librerías necesarias
import pandas as pd
import os

# Cargar los datos en un DataFrame
calidad_aire = pd.read_csv('https://datosabiertos.jcyl.es/web/jcyl/risp/es/medio-ambiente/calidad_aire_historico/1284212629698.csv', sep = ';')

# Mostrar las primeras filas del DataFrame
calidad_aire.head(2)
print("="*100)

# Mostrar la estructura del DataFrame
print(calidad_aire.info())
print("="*100)

# Mostrar un resumen estadístico de las variables numéricas
print(calidad_aire.describe())
print("="*100)
```



```

Fecha CO (mg/m3) NO (ug/m3) NO2 (ug/m3) O3 (ug/m3) PM10 (ug/m3) PM25 (ug/m3) SO2 (ug/m3) Provincia Estación Latitud Longitud Posición
0 2020-12-31 0.6 8.0 16.0 NaN 6.0 NaN 1.0 Burgos Burgos1 42.351111 -3.675556 42.351111111,-3.67555555556
1 2020-12-31 NaN 2.0 6.0 NaN 8.0 NaN 4.0 León C.T.L.R. - Naredo 42.816667 -5.533333 42.816666667,-5.53333333333

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 446014 entries, 0 to 446013
Data columns (total 13 columns):
#   Column              Non-Null Count  Dtype
---  -
0   Fecha              446014 non-null object
1   CO (mg/m3)         101158 non-null float64
2   NO (ug/m3)         415030 non-null float64
3   NO2 (ug/m3)        413497 non-null float64
4   O3 (ug/m3)         275414 non-null float64
5   PM10 (ug/m3)       344579 non-null float64
6   PM25 (ug/m3)       53784 non-null float64
7   SO2 (ug/m3)        356277 non-null float64
8   Provincia          446014 non-null object
9   Estación           446014 non-null object
10  Latitud            445788 non-null float64
11  Longitud           445788 non-null float64
12  Posición           445788 non-null object
dtypes: float64(9), object(4)
memory usage: 44.2+ MB
None

count CO (mg/m3) NO (ug/m3) NO2 (ug/m3) O3 (ug/m3) PM10 (ug/m3) PM25 (ug/m3) SO2 (ug/m3) Latitud Longitud
mean 0.854624 13.225808 21.409154 52.619754 22.694662 13.677172 9.092801 42.151547 -5.178965
std 0.785226 21.970729 19.108434 23.221958 17.919319 15.895495 13.790750 0.665501 1.121920
min 0.000000 -441.000000 0.000000 0.000000 0.000000 0.000000 -791.000000 38.938333 -6.781944
25% 0.300000 2.000000 8.000000 37.000000 11.000000 5.000000 2.000000 41.645556 -6.483889
50% 0.700000 5.000000 16.000000 54.000000 18.000000 9.000000 5.000000 42.542778 -4.909167
75% 1.100000 15.000000 29.000000 68.000000 29.000000 15.000000 11.000000 42.688056 -4.538333
max 25.100000 634.000000 249.000000 999.000000 557.000000 223.000000 364.000000 43.603333 -2.466667

```

Figure 2 - General information about the dataset

With **df** being the name of the object where the DataFrame is stored:

- **df.head():** shows the first row of the DataFrame.
- **df.info():** provides a compact view of the internal structure of the DataFrame, indicating the variable types, the number of non-null values, and the memory usage.
- **df.describe():** shows a statistical summary of the numerical variables in the DataFrame, including minimum, maximum, mean, median, first and third quartiles, and the number of missing values.

Graphical data visualization is a fundamental pillar in the exploratory analysis process. Visual representations not only complement numerical analyses but also reveal crucial aspects of the data that might be imperceptible through purely quantitative methods: behavior patterns, temporal trends, relationships between variables, and potential anomalies emerge clearly through appropriate visualizations. The available arsenal of visual tools is broad and versatile, including histograms for distribution analysis, line charts for temporal evolution, bar charts for categorical comparisons, and pie charts for compositional analyses, among others. Selecting the most suitable type of visualization for each analysis is crucial and can be further explored by consulting this [Data Visualization Guide](#).

Particularly, the **histogram** stands out for **its ability to represent the distribution of numerical variables**. By grouping data into intervals or ‘bins,’ these charts reveal crucial patterns in the underlying structure of the data. The shape of a histogram provides essential information about fundamental statistical characteristics: it can show whether the distribution is symmetrical or has asymmetries (positive or negative skewness), whether it is unimodal or multimodal, whether it approximately follows a normal distribution or fits better to other probabilistic models, among others.

The detailed interpretation of a histogram allows for the identification of various critical aspects for subsequent analysis:

- The central tendency of the data, observable through the location of the peak or peaks of the distribution.
- The dispersion or variability, reflected in the width and spread of the bars.
- The presence of long or short tails, which can indicate the frequency of extreme values.
- Possible discontinuities or gaps in the distribution, which could indicate issues in data collection or important underlying phenomena.
- The existence of outliers, visible as isolated bars at the extremes of the distribution.

Understanding the shape of the distribution is crucial for the selection of subsequent statistical methods, as many techniques assume normality or other specific distributional characteristics. For example, a strongly skewed distribution might require data transformation or the use of non-parametric statistical methods, while a bimodal distribution could suggest the presence of distinct subpopulations in the data that warrant separate analysis.

Next, **histograms are generated for all numerical variables** present in the dataset, after importing the data visualization library **matplotlib**. Through more advanced programming techniques, such as iterating over the dataset columns and automating the generation of subplots, we can optimize the visualization process and obtain all distributions on a single canvas. This programmatic approach is not only more efficient than manually creating individual charts, but it also facilitates the detection of common patterns or significant divergences among the different numerical variables in the dataset.

```
import matplotlib.pyplot as plt
import numpy as np

# Seleccionar solo las columnas numéricas
columnas_numericas = calidad_aire.select_dtypes(include=[np.number]).columns

# Calcular el número de filas y columnas para el subplot
n = len(columnas_numericas)
nrows = 3
ncols = min(n, 3)

# Crear la figura y los subplots
fig, axes = plt.subplots(nrows=nrows, ncols=ncols, figsize=(15, 5*nrows))
fig.suptitle('Distribución de Variables Numéricas', fontsize=16)

# Aplanar el array de ejes en caso de que sea 2D
axes = axes.flatten() if n > 3 else [axes]

# Crear histogramas para cada variable numérica
for i, col in enumerate(columnas_numericas):
    ax = axes[i]
    calidad_aire[col].hist(ax=ax, bins=50, edgecolor='black')
    ax.set_title(f'Distribución de {col}')
    ax.set_xlabel(col)
    ax.set_ylabel('Frecuencia')

# Ocultar subplots vacíos si los hay
for j in range(i+1, len(axes)):
    fig.delaxes(axes[j])
```

```
plt.tight_layout()
plt.show()
```

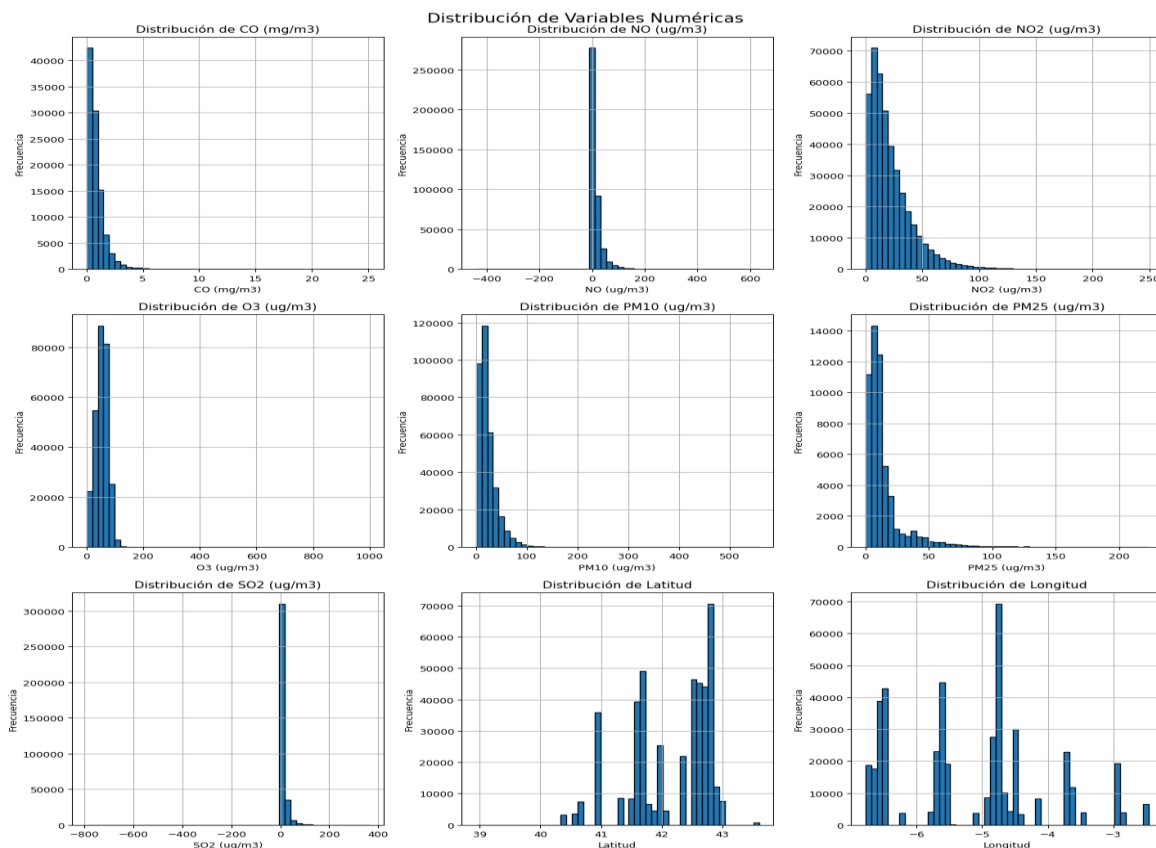


Figure 3 – Numerical variables histograms

In this set of histograms, it is particularly interesting to analyze the distribution of NO₂ ($\mu\text{g}/\text{m}^3$). Its histogram reveals a **clearly asymmetric distribution with a pronounced positive skew**, where most measurements are concentrated at low values (below 50 $\mu\text{g}/\text{m}^3$), but it has a long tail to the right extending up to approximately 250 $\mu\text{g}/\text{m}^3$. This distribution is typical of atmospheric pollutants in urban environments, where periods of relatively low baseline concentrations are combined with occasional episodes of high pollution, possibly associated with peak traffic hours or specific weather conditions. The shape of this distribution suggests that logarithmic transformations might be necessary for subsequent statistical analyses that assume normality, and it also indicates the importance of paying special attention to those extreme values which, although less frequent, could represent critical pollution episodes from a public health perspective.

2.2. ADJUSTMENT OF VARIABLE TYPES



Figure 4 – EDA Second Stage - Adjustment of variable types

After the initial load, it is essential **to verify the correct encoding of the data type for each variable**. An inappropriate data type can compromise subsequent analyses or generate erroneous results. It is necessary to confirm that numerical variables are indeed stored as numbers (whether integers or decimals), while qualitative or categorical variables should be encoded as character strings and contain a finite and well-defined set of categories. This early verification allows for the identification and correction of possible inconsistencies in data typology, such as dates stored as text or categories incorrectly encoded as numerical values.

The usual types of variables that our data table can contain may be:

- **Numeric:** stores numbers that can be decimal or integer.
- **Character:** holds text strings.
- **Categorical:** contains a limited number of values or categories of information.
- **Logical or Boolean:** represents binary variables that can only take two values: True and False or 0 and 1; they can result from a comparison or condition of other variables present in the dataset.
- **Date:** stores specific time intervals.

Correctly typing the variables in a dataset is not just a matter of format, but **it is essential to ensure the integrity and effectiveness of subsequent analyses**. Proper classification of data types adheres to fundamental principles of data analysis.

- Temporal variables (dates) require special treatment to capture the sequential nature and cyclical properties of time.
- Categorical variables must preserve the hierarchical or nominal structure of their categories.
- Numerical data must maintain their mathematical and statistical properties.

2.2.1. EXPERIMENT

With the `df.info()` function, we can perform an initial inspection of the data types assigned to each variable. In our dataset, **we find various variables where the automatically assigned data type does not correspond to the intrinsic nature of the information they contain**, as is the case with the variables `Fecha`, `Provincia` and `Estación`. These three variables have been generically encoded as type object, which significantly limits their analytical utility:

- Variable `Fecha` requires a conversion to date type (`datetime`) to allow temporal operations such as interval calculations, period aggregations, or seasonality analysis. The transformation

using `pd.to_datetime()` not only changes the format but also enables the entire ecosystem of temporal analysis in Python.

- Variables `Provincia` y `Estación` represent categorical variables, whose conversion to `type('category')` optimizes both memory usage and computational performance. This type of data is especially relevant in exploratory analysis because:
 1. Facilitates the detection of incorrect categories or anomalies.
 2. Allows efficient grouping and filtering operations.
 3. Enables specific analyses for categorical variables such as contingency tables.
 4. Optimizes the generation of categorical visualizations.

The procedure to follow is to readjust the types of these variables in order to subsequently perform the necessary operations, analyses, and graphical representations.

```
# Ajustar el tipo de la variable Fecha
calidad_aire['Fecha'] = pd.to_datetime(calidad_aire['Fecha'], errors='coerce')

# Ajustar el tipo de la variable Provincia
print(calidad_aire['Provincia'].unique())
calidad_aire['Provincia'] = calidad_aire['Provincia'].astype('category')

# Ajustar el tipo de la variable Estación
print(calidad_aire['Estación'].unique())
calidad_aire['Estación'] = calidad_aire['Estación'].astype('category')
```

```
['Burgos' 'León' 'Palencia' 'Salamanca' 'Valladolid' 'Soria' 'Zamora' 'Ávila' 'Segovia' 'Madrid']
['Burgos1' 'C.T.L.R. - Naredo' 'Carracedelo' 'La Robla' 'Tudela Veguín-Tudela Veguín' 'Valderas' 'Guardo' 'Hontoria 1 - Poblado'
'Renault4' 'El Maillo' 'Salamanca5' 'Arco de ladrillo II' 'La rubia II' 'Medina del Campo' 'Michelin2' 'Puente Poniente-Mª Luisa Sánchez'
'Vega Sicilia' 'Aranda de Duero 2' 'Miranda de Ebro2' 'C.T.L.R. - Cuadros' 'C.T.L.R. - Ventosilla' 'Lario' 'Leon1' 'Otero'
'Toral de los Vados' 'C.T.G. - Villalba' 'Palencia 3' 'Muriel de la Fuente' 'Soria' 'Renault3' 'Zamora 2' 'Ávila II' 'Burgos4'
'Medina de Pomar' 'Leon 4' 'Ponferrada4' 'C.T.G. - Compuerto' 'Hontoria 2 - Venta de Baños' 'Salamanca6' 'Segovia 2' 'Michelin1'
'Renault1' 'Renault2' 'VALLADOLID SUR' 'C.T.Compostilla-Congosto' 'C.T.Compostilla-Cortiguera' 'C.T.Compostilla-Villaverde'
'C.T.Compostilla-Compostilla' 'C.T.Compostilla-Santa Marina' 'NH3' 'Miranda de Ebro1' 'Puente Regueral' 'San Martín de Valdeiglesias'
'C.T.Anllares - Palacios del Sil' 'C.T.Anllares - Susaño' 'C.T.Anllares - Hospital del Sil' 'C.T.Anllares - Anllares'
'C.T.Anllares - Lillo' 'C.T.Compostilla-San Miguel' 'C.T.Compostilla-Sancedo' 'C.T.Compostilla-Cueto' 'Burgos5' 'Salamanca4' 'Leon3'
'Labradores II' 'Cementerio' 'Velilla del Río Carrion' 'Ponferrada5' 'Ponferrada1' 'Salamanca2' 'Venta de Baños' 'Santa Teresa' 'Leon2'
'Burgos3' 'C.T.Anllares - Anllarinos' 'C.T.Anllares - Páramo del Sil' 'C.T.Anllares - Sorbeda' 'C.T.L.R. - La Robla' 'Zamora' 'Ávila'
'Segovia' 'Aranda de Duero' 'Palencia2' 'Salamanca3' 'Burgos2' 'Ponferrada3' 'Ponferrada2' 'Miranda de Ebro3' 'Palencia1' 'Salamanca1'
'Arco Ladrillo I']
```

Figure 5 - Unique categories of the transformed categorical variables

2.3. DETECTION AND TREATMENT OF MISSING DATA



Figure 6 -EDA Third Stage - Detection and Treatment of Missing Data

2.3.1. DETECTION OF MISSING DATA

The **presence of missing data**, also known as missing values or NaN (Not a Number) in Python, is a common situation in many datasets and **can pose a significant challenge in data analysis**. These missing values can arise for various reasons, such as errors during data transcription, issues in the collection process, or even because certain values were not available at the time of measurement.

Proper management of missing data is crucial to ensure the quality and reliability of statistical analysis. Missing data can distort analysis results, affect the accuracy of predictive models, and alter graphical visualizations, leading to incorrect or misleading interpretations. For example, if not handled properly, missing data can bias regression results or decrease a model's predictive capability.

2.3.1.1. EXPERIMENT

In Python, we can use the pandas and numpy libraries to work with missing data. Below, we show some useful functions to detect missing values:

```
#Sección 1

# Devuelve un DataFrame booleano
calidad_aire.isna()

# Devuelve True si hay al menos un valor ausente
calidad_aire.isna().any().any()

# Devuelve el número total de NaN que presenta el DataFrame
print(calidad_aire.isna().sum().sum())

#Sección 2

# Devuelve el % de valores perdidos
print(calidad_aire.isna().mean().mean())

# Detección del número de valores perdidos en cada una de las columnas
calidad_aire.isna().sum()

# Detección del % de valores perdidos en cada una de las columnas
calidad_aire.isna().mean().round(2)
```

```

1163037
0.20058649418041727
Fecha          0.00
CO (mg/m3)     0.77
NO (ug/m3)     0.07
NO2 (ug/m3)    0.07
O3 (ug/m3)     0.38
PM10 (ug/m3)   0.23
PM25 (ug/m3)   0.88
SO2 (ug/m3)    0.20
Provincia      0.00
Estación       0.00
Latitud        0.00
Longitud       0.00
Posición       0.00
dtype: float64

```

Figure 7 - Results of the Missing Values Analysis

This code uses several pandas functions:

- In the first section, `isna()` to generate a boolean DataFrame indicating the presence of missing values, `any()` to check if there is at least one NaN in the DataFrame, and `sum()` to count the total number of NaNs.
- Additionally, the percentage of missing values is calculated using `mean()`. Finally, the number and percentage of missing values for each column are obtained.

A thorough analysis of the missing values in our dataset reveals a situation that requires special attention: the DataFrame "calidad_aire" **has 1,163,037 missing values, which represents a significant 20% of the total observations**. This percentage is the same as that obtained in the analysis of the guide in R, with 116,281 missing values. This fact confirms a non-random pattern in data loss that could be related to the availability or capacity of the measurement systems.

The distribution of these missing values is not uniform among the variables, with a notable absence of data in the parameters of CO (mg/m³) and PM25 (µg/m³), with 77% and 88% missing values respectively. The magnitude and nature of these missing data pose a significant methodological challenge that must be carefully addressed to ensure the validity and usefulness of subsequent analysis. Managing these missing values will require a strategy that balances preserving data integrity with the need to maintain a sufficiently complete dataset for effective analysis and reuse.

2.3.2. TREATMENT OF MISSING DATA

Handling missing values is a crucial aspect of data preparation. There are various strategies to deal with missing data, each with its advantages and disadvantages. Below are some of the most common techniques:

- **Fill with descriptive statistics:**
 - **Mean:** replace missing values with the mean of the variable. This is useful when the data is normally distributed and there are not many missing values
 - **Median:** using the median is a good option if the data is skewed or contains outliers, as it is less sensitive to these extremes than the mean.
 - **Mode:** replacing with the most frequent value is suitable for categorical variables.
- **Fill by adjacent values:**
 - **Forward or backward imputation:** Fill in the missing values with the value from the previous or next row or column. This method is useful in time series where the data are often correlated over time.
- **Fill with zeroes:**
 - **Zero assignment:** For numerical values, filling with zero can be simple, but it is generally discouraged as it can introduce significant bias and alter the results.
- **Delete rows:**
 - **Rows deletion:** If the missing data are present in a small number of rows and the dataset is large, these rows can be deleted. This technique can be useful to avoid imputation, but care must be taken not to lose valuable information.
- **Imputation with machine learning algorithms:**
 - **Predictive models:** Use algorithms to predict the missing values based on other data. This technique can provide more accurate imputations, but it is more complex and computationally intensive.
- **Delete variables:**
 - **Variables with a high rate of missing data:** In some cases, it may be appropriate to remove variables that have more than 50% missing data, especially if the variable is not very relevant to the analysis.

The selection of the appropriate treatment technique depends on the type of data, the amount and pattern of missing data, and the context of the analysis. Although mean imputation is a common technique, it is not always the most suitable. It is essential **to evaluate how each method can affect the results and the quality** of the final analysis.

Additionally, it is important to **carefully document any decisions made** in the treatment of missing data. A rigorous EDA design includes traceability of these processes to evaluate their impact and make adjustments if inconsistencies or weaknesses are detected in later stages of the analysis.

2.3.2.1. EXPERIMENT

As an example of applying the listed options, the first treatment to be performed on the missing data is **the elimination of the two variables that present a percentage higher than 50%**, as such a high number

of NaNs can produce errors or distort subsequent analyses by not using the rows that present NaNs (in this case, more than 50% of the observations would not be used). Before this, a copy of the original dataset is saved, which will be used in point 3.

```
# Guardar copia del dataset original
calidad_aire_original = calidad_aire.copy()

# Eliminación de las variables que presentan un % de NaN superior al 50%
calidad_aire = calidad_aire.loc[:, calidad_aire.isna().mean() < 0.5]
print(f" Tras esta operación, contamos con {len(calidad_aire.columns)} columnas")

Tras esta operación, contamos con 11 columnas
```

Figure 8 - Results of the Missing Data Treatment

Continuing with the example, we will replace the missing values in the DataFrame for the remaining variables with the mean of each column, so as not to lose significant information and to ensure that subsequent analyses are not altered.

```
# Seleccionamos las variables numéricas
columnas_numericas = calidad_aire.select_dtypes(include=[np.number]).columns

# Calculamos la media para cada una de las variables numéricas sin tener en cuenta los NaN
cols_mean = calidad_aire[columnas_numericas].mean()

# Sustituimos los valores NaN por la media correspondiente a cada variable
calidad_aire[columnas_numericas] = calidad_aire[columnas_numericas].fillna(cols_mean)
```

It is important to note that the treatment performed, although valid as an illustrative example, represents a **simplified approach** to the problem of missing values. In a more thorough analysis of air quality data, additional aspects should be considered such as:

- **The temporal nature of the measurements:** Air pollutants usually exhibit daily and seasonal patterns, so simple mean imputation might not capture these cyclical variations. A more sophisticated approach could consider imputation based on moving averages or values from equivalent time periods.
- **Spatial correlation:** Since the data come from different measurement stations, it would be relevant to consider the geographical proximity between stations for the imputation of missing values, as nearby stations tend to record similar levels of pollution.
- **The meteorological context:** The concentration of pollutants is strongly influenced by meteorological variables such as temperature, precipitation, wind direction, and wind speed. A more robust imputation method could incorporate these variables as predictors.
- **The pattern of missingness:** Before removing variables with a high percentage of missing values, it would be advisable to analyze whether they follow any systematic pattern (e.g., specific sensor failures or maintenance periods) that could provide relevant information about the data quality.

These considerations underscore the importance of adapting the techniques for handling missing values to the specific context of the problem and the nature of the data analyzed.

2.4. DETECTION AND TREATMENT OF OUTLIERS



Figure 9 -EDA Fourth Stage -Outliers Identification

An **atypical value** or **outlier** represents an **observation that exhibits a significant deviation from the general pattern of behavior of the rest of the data**. These extreme observations can arise for various reasons: from errors in measurement or data recording to real but extraordinary phenomena that deserve special attention. Their importance in exploratory analysis is crucial, as they can exert a disproportionate influence on descriptive statistics, distort relationships between variables, and compromise the validity of subsequent statistical models. For example, in air quality data, an outlier could represent both a measurement error and a real episode of severe pollution, making its identification and contextualized analysis especially relevant before making decisions about its treatment. Proper management of these values requires a balance between preserving potentially valuable information and maintaining the robustness of the statistical analysis.

The most common approach to handling outliers is to reduce their potential influence on analyses. Below are some strategies that can be considered:

- **Robust statistical methods:** There are robust statistical techniques designed to minimize the impact of outliers on results. These methods adjust the analysis to be less sensitive to outliers, thus preserving the integrity of the results.
- **Removal of outliers:** Removing outliers can be appropriate in some cases, but it should be done carefully. Before discarding an outlier, it is essential to verify whether the value is the result of a measurement error or a problem in the dataset construction.
- **Substitution of outliers:** replacing outliers with the mean or median, for example. Although this practice may seem like a simple solution, it can alter the distribution and variance of the data, introducing bias into the analysis.

“If it is decided to remove or replace outliers, it is prudent to repeat the analyses with both the original values and the modified data. This allows for observing the real impact of the outliers on the results. If the difference is minimal, it may be reasonable to proceed with the removal or replacement. However, if the impact is considerable, **any decision should be adequately justified**.”

Regardless of the approach taken, as with the treatment of missing data, it is also crucial to document all decisions made during the process of handling outliers. This ensures that **other analysts can understand the transformations performed on the dataset** and allows for proper traceability throughout the Exploratory Data Analysis.

The following shows how outliers can be detected and removed, assuming it can be justified that the values are measurement errors or issues arising from data ingestion. The goal is to prevent these values from distorting future statistical analyses.

2.4.1. OUTLIERS DETECTION

To demonstrate the process, we must distinguish between two types of treatment based on the type of variables: continuous or discrete and categorical.

2.4.1.1. CONTINUOUS VARIABLES

To demonstrate the process of outlier detection in a continuous variable, we will use the numerical variable 'O3 ($\mu\text{g}/\text{m}^3$)' as an example. The process is exactly the same for the rest of the numerical variables in the table.

First, we generate a histogram to understand the frequency distribution of the variable under study:

```
plt.hist(calidad_aire['O3 (ug/m3)'], bins=100, range=(0, 150), color='blue', edgecolor='black')
plt.title('Distribución de O3 (ug/m3)')
plt.xlabel('O3 (ug/m3)')
plt.ylabel('Frecuencia')
plt.xlim(0,150)
plt.tight_layout()
plt.show()
```

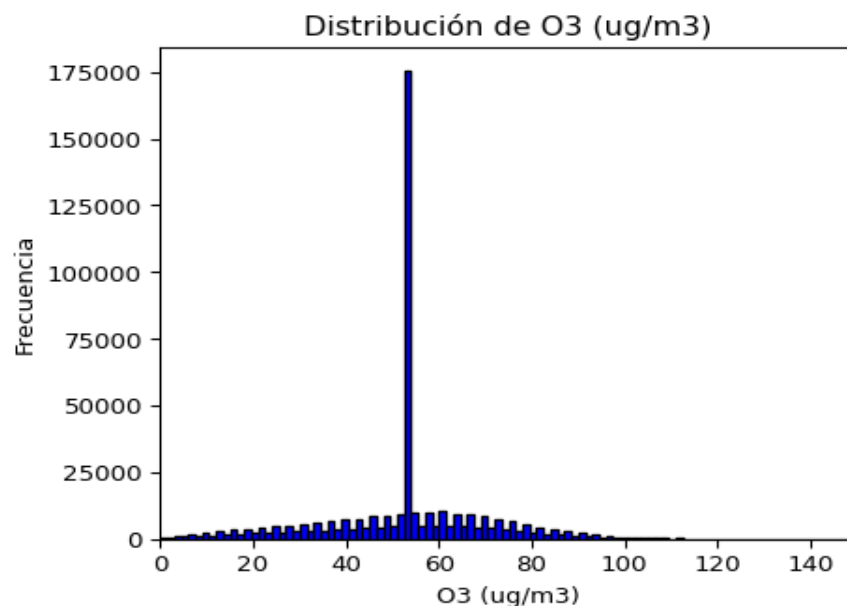


Figure 10 - Variable 'O3 ($\mu\text{g}/\text{m}^3$)' Histogram

The analysis of the O3 histogram reveals a positively skewed distribution characteristic of atmospheric pollutant measurements, where most observations are concentrated in the range of 0 to 100 $\mu\text{g}/\text{m}^3$. Observations exceeding this threshold show a markedly lower frequency, suggesting the presence of

potential outliers. However, in the context of air quality, these high values could represent actual episodes of high ozone concentration, typically associated with specific meteorological conditions such as high solar radiation and elevated temperature, especially during the summer months. To more appropriately detect the presence of outliers, we will use the most suitable representation for this task: a box plot.

Box plots provide a **visual representation that describes the dispersion and symmetry of the data** by observing the quartiles (division of the distribution into four parts delimited by the values 0.25, 0.50, and 0.75). These plots consist of three components:

1. **Interquartile range (IQR):** It represents 50% of the data, ranging from the 25th percentile of the distribution (Q1) to the 75th percentile (Q3). Inside the box, we find a line indicating the 50th percentile of the distribution (Q2), the median. The box provides an idea of the distribution's dispersion based on the separation between Q1 and Q3, as well as whether the distribution is symmetric around the median or skewed to one side.
2. **Whiskers:** they extend from both sides of the ends of the box and represent the ranges of the 25% of values in the lower part ($Q1 - 1.5 \text{ IQR}$) and the 25% of values in the upper part ($Q3 + 1.5 \text{ IQR}$), excluding outliers.
3. **Outliers:** This representation identifies as outliers those observations that have values lower or higher than the limits of the plot (lower limit: $Q1 - 1.5 \text{ IQR}$ and upper limit: $Q3 + 1.5 \text{ IQR}$).

To obtain the necessary statistics for the plot representation, we will use the `boxplot()` function from `seaborn`, another popular data visualization library:

```
import seaborn as sns
# Estadísticas necesarias para reproducir el gráfico de cajas y bigotes
Q1 = calidad_aire['O3 (ug/m3)'].quantile(0.25)
Q3 = calidad_aire['O3 (ug/m3)'].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

print(f"Estadísticas para O3:")
print(f"Q1 - 1.5IQR = {lower_bound:.2f}")
print(f"Q1 = {Q1:.2f}")
print(f"Mediana = {calidad_aire['O3 (ug/m3)'].median():.2f}")
print(f"Q3 = {Q3:.2f}")
print(f"Q3 + 1.5IQR = {upper_bound:.2f}")
print(f"Número de observaciones: {len(calidad_aire['O3 (ug/m3)'])}")
print(f"Número de outliers: {sum((calidad_aire['O3 (ug/m3)'] < lower_bound) | (calidad_aire['O3 (ug/m3)'] > upper_bound))}")

# Construcción del gráfico de cajas y bigotes
plt.figure(figsize=(10, 6))
sns.boxplot(x=calidad_aire['O3 (ug/m3)'])
plt.title('Gráfico de cajas y bigotes para O3 (µg/m3)')
plt.xlabel('O3 (µg/m3)')
plt.show()
```

```

Estadísticas para O3:
Q1 - 1.5IQR = 31.50
Q1 = 48.00
Mediana = 52.62
Q3 = 59.00
Q3 + 1.5IQR = 75.50
Número de observaciones: 446014
Número de outliers: 91163

```

Figure 11 - Statistics on the distribution of the variable 'O3 (ug/m³)'

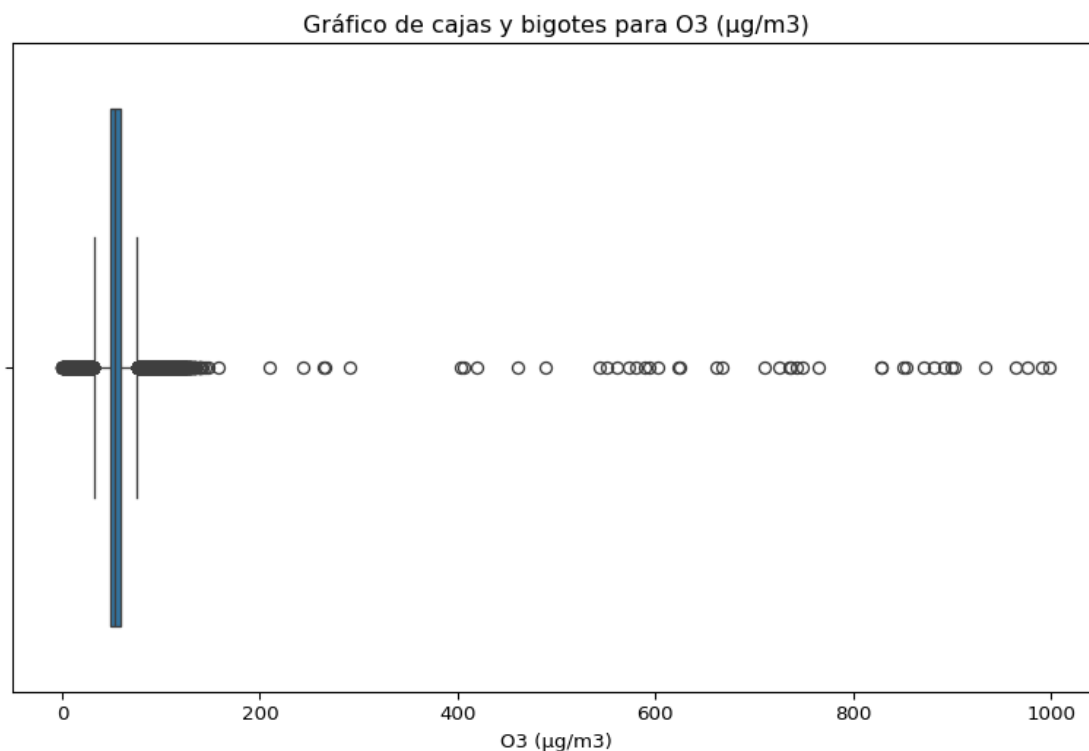


Figure 12 - Box plot of the variable 'µg/m³'

A detailed analysis of the O3 variable **reveals interesting patterns in its distribution**. The descriptive statistics show a median of 52.62 µg/m³, with an interquartile range (IQR) from 31.50 µg/m³ (Q1) to 75.50 µg/m³ (Q3). The box plot illustrates a significant presence of outliers above the upper limit, specifically identifying **91,163 observations that exceed this threshold** out of a total of 446,014 measurements (approximately 20% of the data).

It is important to note that this analysis represents an illustrative example of basic outlier identification using simple statistical methods. In a comprehensive air quality study, the treatment of these values would require additional domain-specific considerations and a deeper analysis of the causes of these deviations. However, for the educational purposes of this guide, this approach serves to demonstrate the basic concepts of outlier analysis.

2.4.1.2. CATEGORICAL VARIABLES

Outlier detection in categorical variables requires a different approach than that used for numerical variables, focusing **on identifying unusual or inconsistent categories within the problem domain**. For this analysis, visualization using bar charts or frequency diagrams is particularly useful, as it allows for identifying both the distribution of expected categories and the possible presence of anomalous categories that could represent coding errors, data collection issues, or special cases that require attention.

In our case, we will use the variable 'Provincia' as an illustrative example of this process. This choice is particularly suitable because the valid categories are clearly defined (the provinces of Castilla y León), which facilitates the identification of possible anomalies such as spelling errors, variations in writing format, or the presence of provinces that do not belong to the autonomous community. This same analysis procedure can and should be applied to any categorical variable in the dataset, adapting the validation criteria according to the specific nature of each variable.

In Python, we can use the matplotlib or seaborn library to create bar charts and visualize the distribution of categories.

```
# Número de categorías que presenta la variable Provincia
categoria_counts = calidad_aire['Provincia'].value_counts()

# Construcción del gráfico de barras para la variable Provincia
plt.figure(figsize=(10, 6))
sns.barplot(x=categoria_counts.index, y=categoria_counts.values, palette='Blues')
plt.xlabel('Provincias')
plt.ylabel('Nº observaciones')
plt.xticks(rotation=30)
plt.title('Distribución de la variable Provincia')
plt.show()
```

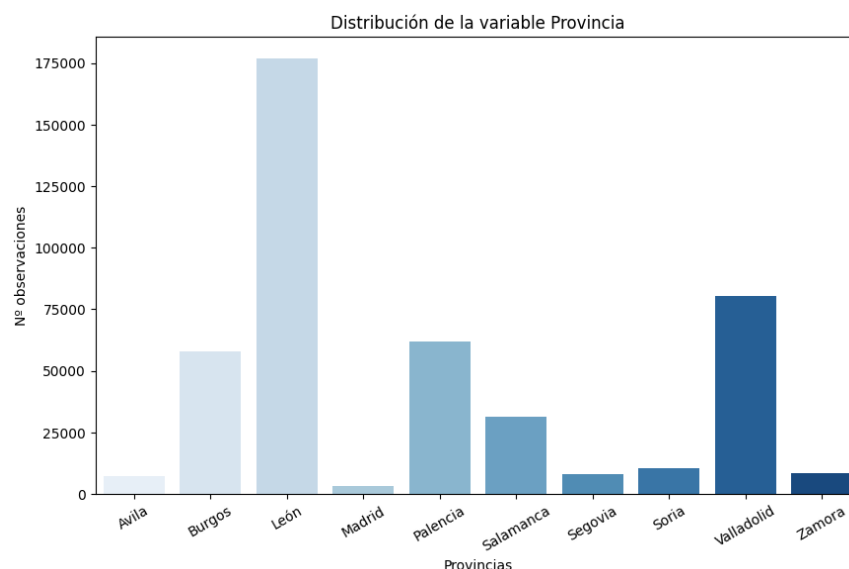


Figure 13 - Bar chart of the categorical variable 'Provincia'

Based on the exploratory analysis, we can deduce that **the category Madrid is an outlier within the variable 'Provincia'**, as it does not belong to Castilla y León. In the next section, we will proceed to remove this category from our data.

2.4.2. OUTLIER REMOVAL

2.4.2.1. CONTINUOUS VARIABLES

Once the outliers have been identified, we proceed to remove them. One way to remove outliers from a numerical variable is to **generate a new table**.

```
# Se genera una nueva tabla que no contiene los valores identificados como atípicos
calidad_aire_NoOut = calidad_aire[(calidad_aire['O3 (ug/m3)'] >= lower_bound) &
(calidad_aire['O3 (ug/m3)'] <= upper_bound)]

# Construcción de los gráficos de cajas y bigotes
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))

sns.boxplot(x=calidad_aire['O3 (ug/m3)'], ax=ax1)
ax1.set_title('O3 (ug/m3) con outliers')
ax1.set_xlabel('O3 (ug/m3)')

sns.boxplot(x=calidad_aire_NoOut['O3 (ug/m3)'], ax=ax2)
ax2.set_title('O3 (ug/m3) sin outliers')
ax2.set_xlabel('O3 (ug/m3)')

plt.tight_layout()
plt.show()
```

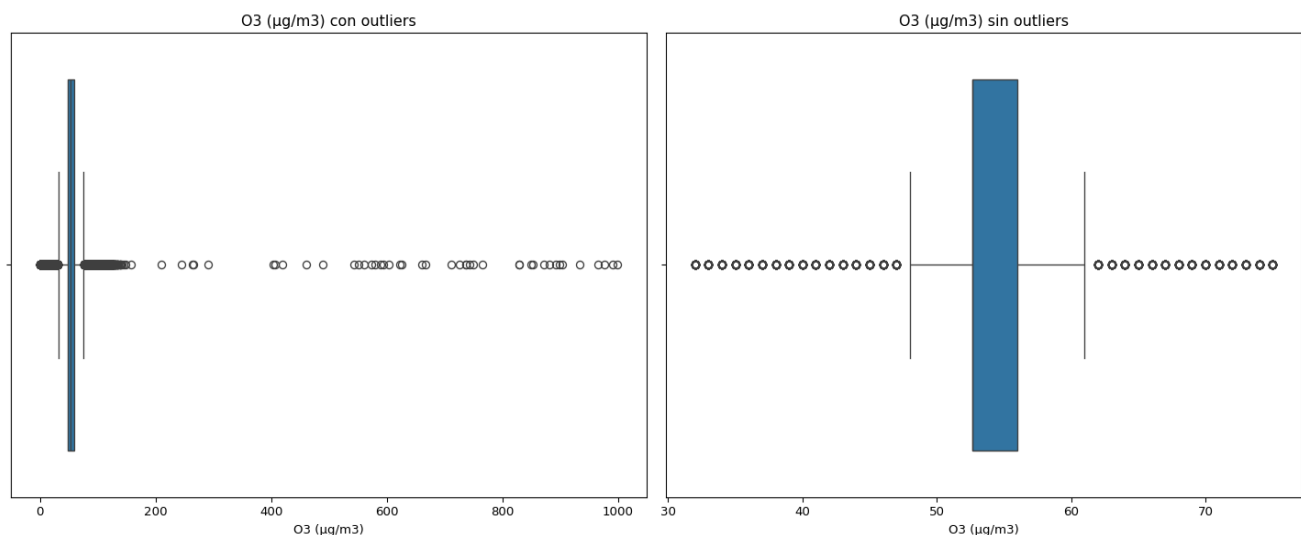


Figure 14 - Box plot of the variable 'O3 ($\mu\text{g}/\text{m}^3$)' before and after outlier removal

The comparison of the box plots before and after outlier removal reveals significant changes in the distribution of the O3 ($\mu\text{g}/\text{m}^3$) variable. The post-removal plot shows a more compact and symmetric distribution, with a considerably narrower range oscillating approximately between 30 and 70 $\mu\text{g}/\text{m}^3$. This change in scale allows for a better appreciation of the central structure of the data, where both the median and the quartiles are more clearly visualized.

2.4.2.1. CATEGORICAL VARIABLES

In Python, we can use the **drop()** method to remove rows containing the outlier category and **astype('category').cat.remove_unused_categories()** to ensure the category is removed from the categorical data type.

```
# Eliminamos las filas que pertenecen al factor "Madrid"
calidad_aire_SM = calidad_aire[calidad_aire['Provincia'] != 'Madrid'].copy()

# Eliminamos el factor "Madrid"
calidad_aire_SM['Provincia'] = calidad_aire_SM['Provincia'].astype('category').cat.remove_unused_categories()

# Verificamos la eliminación de la categoría "Madrid"
print(calidad_aire_SM['Provincia'].cat.categories)

Index(['Avila', 'Burgos', 'León', 'Palencia', 'Salamanca', 'Segovia', 'Soria', 'Valladolid', 'Zamora'],
      dtype='object')
```

Figure 15 - Unique categories of the variable 'Provincia' after outlier removal

2.5. CORRELATION ANALYSIS BETWEEN VARIABLES



Figure 16 – EDA Fifth Stage – Variables Correlation

Correlation (r) measures the linear relationship between two or more variables, reflecting both the strength and direction of their relationship. In simple terms, correlation tells us if two variables change together and how they do so:

- **Positive Correlation:** if one variable increases and the other also increases, they are said to be positively correlated. A value of (r) close to +1 indicates a strong positive relationship.”
- **Negative Correlation:** if one variable increases while the other decreases, they are negatively correlated. A value of (r) close to -1 indicates a strong negative relationship.
- **No Correlation:** a value of (r) close to 0 suggests that there is no clear linear relationship between the variables.

It is important to note that Pearson correlation, being the most common, is not the only measure of correlation available. There are alternatives such as:

- **Spearman Correlation**, more robust against outliers and useful for non-linear monotonic relationships.
- **Kendall Correlation**, especially useful for small samples and when the data do not follow a normal distribution.

- **Rank-based correlation measures**, which can capture non-linear relationships between variables.

Correlation analysis can be useful in several aspects:

- **Identification of redundancy**: it can help identify redundant variables in a dataset. If two variables are highly correlated, one of them could be eliminated without losing significant information, simplifying data analysis and processing.
- **Simplicity in analysis**: in the context of EDA, understanding the correlations between variables can guide the selection of variables for predictive models and other statistical analyses.
- **Relationship with PCA**: correlation analysis is related to techniques such as Principal Component Analysis (PCA). PCA uses the correlation matrix to transform the original variables into a new set of variables, called principal components, which capture the greatest variability in the data.

In the specific context of environmental data and air quality, correlation analysis becomes especially relevant by allowing a holistic understanding of the interactions between pollutants and their environment. This technique is fundamental for identifying **relationships between different pollutants** that share common emission sources, while also facilitating the understanding of how **meteorological conditions influence** their concentrations. Additionally, this analysis allows for the detection of **temporal and spatial patterns in the distribution of pollutants**, providing a solid basis for interpreting atmospheric dynamics and effectively managing air quality.

It is crucial to remember that **correlation does not imply causation**. Although two variables may be correlated, it does not necessarily mean that one causes changes in the other. Correlation simply indicates an association and does not establish a direct causal relationship.

Correlation analysis is a **fundamental tool in multivariate data exploration**, but it has limitations that must be considered, such as coefficients significantly distorted by the presence of outliers or the **potential omission of non-linear relationships and complex patterns**. To address these limitations, it is essential to complement the analysis with additional tools such as scatterplot visualizations to detect non-linear patterns, **stratified analysis by subgroups** to identify possible heterogeneities in relationships, or the implementation of temporal or spatial cross-validation techniques to confirm the stability of identified correlations. This **comprehensive and multidimensional approach** allows for a more robust and reliable understanding of the relationships between variables in a professional and complete context.

2.5.1. EXPERIMENT

In this section, we will calculate the **correlation matrix for the numerical variables** and represent it graphically.

```
num_variables = calidad_aire.select_dtypes(include=[np.number])

# Calculamos la matriz de coeficientes de correlación entre las variables numéricas
correlacion = num_variables.corr()

# Configuración del gráfico de correlación
plt.figure(figsize=(10, 8))

# Gráfico de correlaciones utilizando un mapa de calor
sns.heatmap(correlacion, annot=True, cmap='coolwarm', center=0, square=True, linewidths=.5, cbar_kws={"shrink": .5})
```

```
plt.title('Matriz de correlaciones entre variables')
plt.show()
```

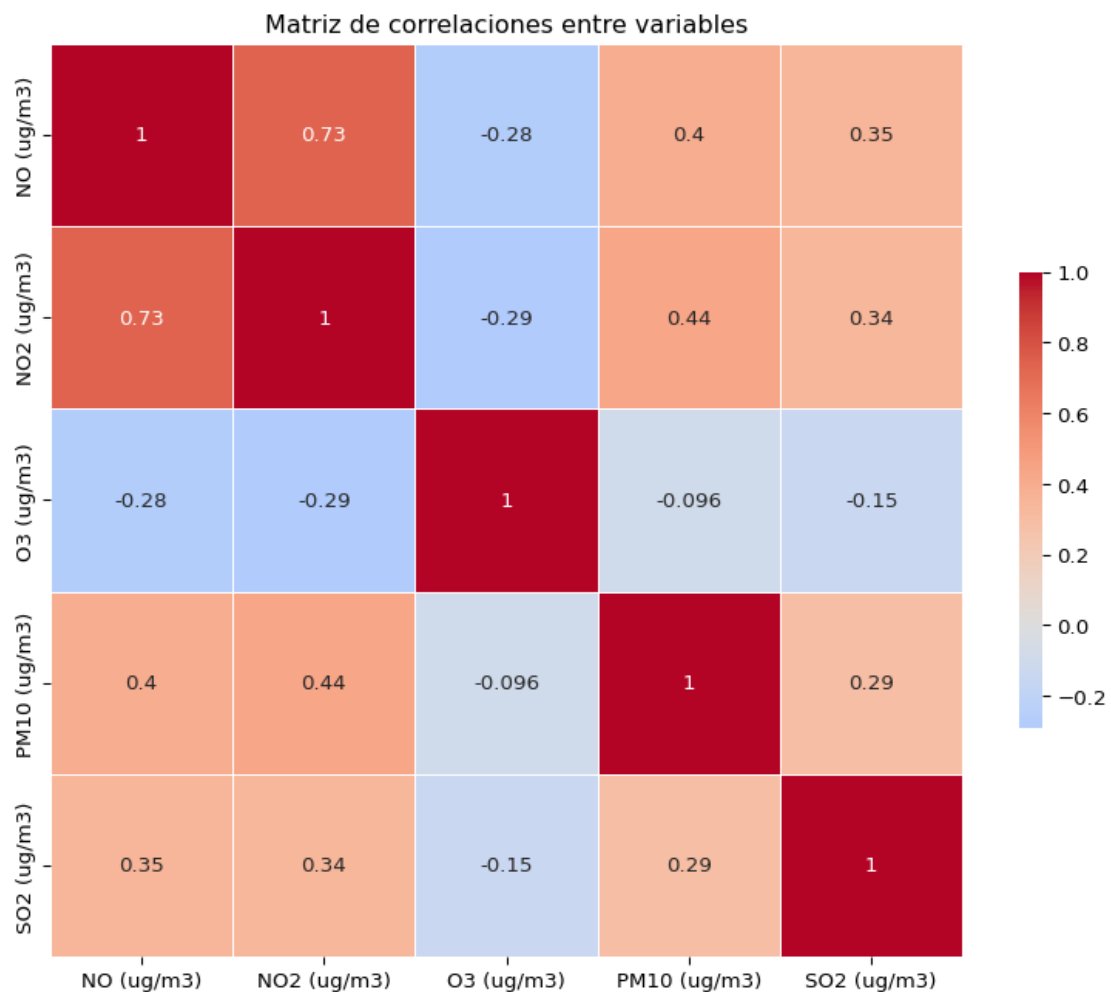


Figure 17 - Correlation matrix of numerical variables

The visualization of correlations between numerical variables has been implemented using a **heatmap**, an effective tool that allows representing the correlation matrix with an intuitive color code, where more intense shades indicate stronger correlations.

The analysis of the correlation matrix reveals several interesting patterns in our air quality dataset:

The most significant correlations are observed between NO ($\mu\text{g}/\text{m}^3$) y NO₂ ($\mu\text{g}/\text{m}^3$), with a coefficient of $r=0.73$, suggesting a strong positive association. This relationship was expected since both pollutants belong to the nitrogen oxides (NO_x) family and share common emission sources, mainly related to combustion processes.

On the other hand, **moderate positive correlations** are identified between PM10 ($\mu\text{g}/\text{m}^3$) and several pollutants: with NO ($r=0.4$) and NO₂ ($r=0.44$). These associations could indicate a common contribution from emission sources, possibly related to urban traffic or industrial processes.

The rest of the variables show lower correlation coefficients. This is reflected in the graph, where the squares representing these correlations are closer to white in color. For example, we could infer that the variables O_3 ($\mu\text{g}/\text{m}^3$) y PM_{10} ($\mu\text{g}/\text{m}^3$) are relatively independent, as their correlation coefficient is low.

These observed correlations provide a basis for understanding the interrelationships between pollutants, although as mentioned earlier, they should be interpreted with caution and in the specific context of urban air quality.

3. AUTOMATED EXPLORATORY DATA ANALYSIS

In today's world, where datasets are increasingly large and complex, the automation of Exploratory Data Analysis has become a very useful tool for data scientists. Python libraries offer efficient solutions for automatically generating EDA reports and visualizations, saving time and providing a quick and comprehensive overview of the data.

An automated EDA provides multiple advantages:

- **Efficiency and speed:** automated EDA allows processing large volumes of data in a short time, generating detailed reports with statistical metrics, visualizations, and correlation analysis automatically.
- **Comprehensive overview:** automated EDA tools provide a panoramic view of the data, including statistical summaries, distributions, relationships between variables, and possible anomalies, facilitating the identification of key patterns and trends.
- **Early problem detection:** automated EDA can help identify issues in the data, such as outliers, missing data, or biases, at an early stage of the analysis, allowing for informed decisions about data preprocessing and cleaning.

However, it is essential that this tool is used considering the following considerations:

- **Interpretation of results:** although automated EDA provides a quick overview, it is essential that the data scientist interprets the results with judgment and knowledge of the problem's context.
- **Personalization:** Automated AED tools offer customization options to tailor reports and visualizations to the specific needs of the analysis.
- **Limitations:** Automated AED may not be suitable for all types of data or analysis. In some cases, a more in-depth and personalized exploratory analysis may be necessary.

3.1. EXPERIMENT

Next, we will use [YData Profiling](#), a library that allows generating interactive AED reports with visualizations, descriptive statistics, correlation analysis, and outlier detection. The report generated with YData Profiling includes:

- Executive summary with general information about the dataset, such as the number of rows and columns, data types, unique values, missing values, etc.

- Univariate analysis of each column, with visualizations and descriptive statistics such as mean, median, standard deviation, minimum and maximum values, distribution, etc.
- Bivariate analysis, with correlation matrices, scatter plots, and analysis of the relationship between variables.
- Detection of outliers and anomalies in the data
- Recommendations and suggestions for improving the dataset.

The following code block shows how to generate an automated AED report with YData Profiling on the `calidad_aire` dataset. This report is generated in the working directory of the chosen execution environment and can be downloaded from the [github repository](#).

```
!pip install setuptools #instalación de paquetes y dependencias
!pip install --upgrade ydata-profiling
!pip install ipywidgets

from ydata_profiling import ProfileReport
report = ProfileReport(calidad_aire_original, title='EDA automático')
report_file = 'reporte_calidad_aire.html'
report.to_file(report_file)
```

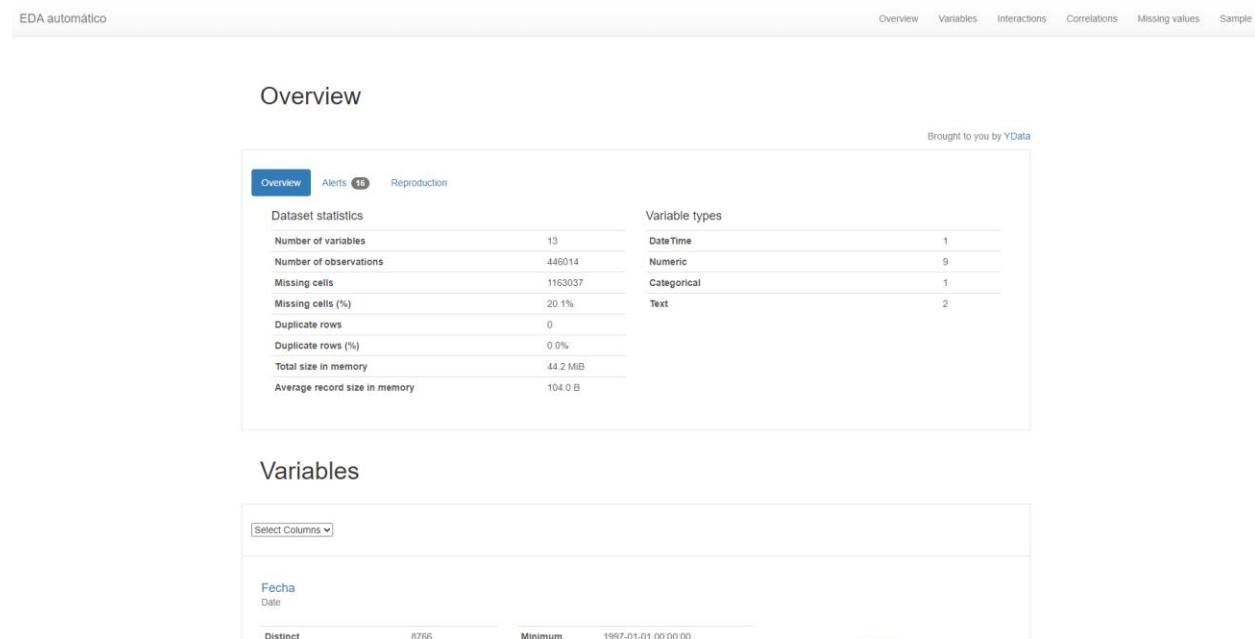


Figure 18 - General summary of the dataset generated by 'ydata-profiling'

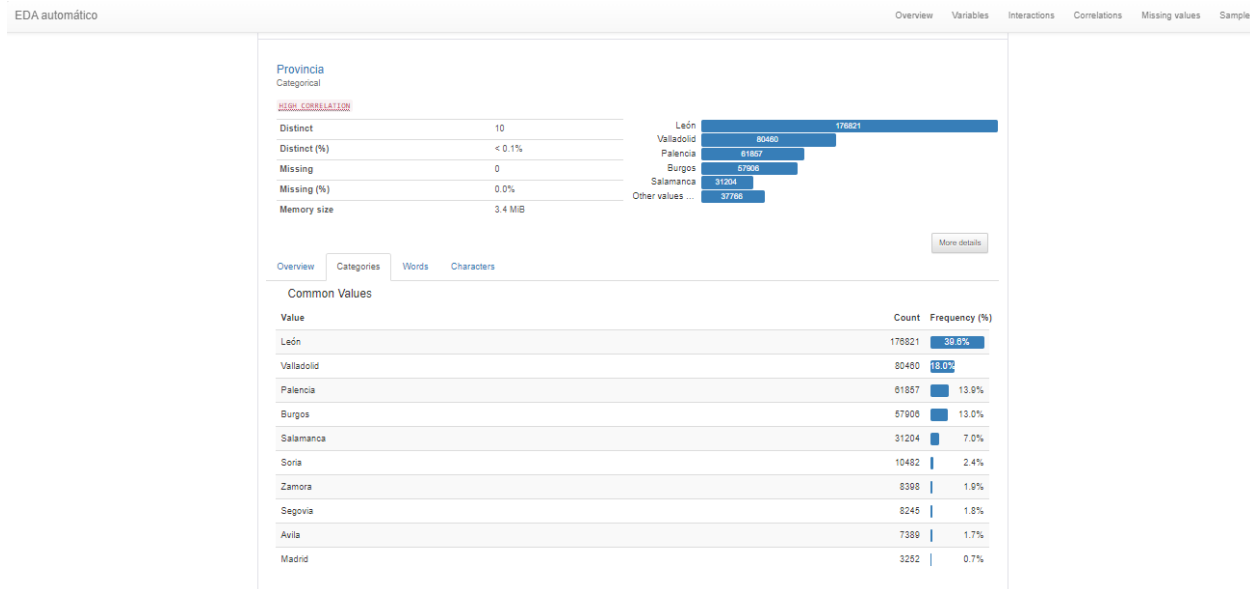


Figure 19 - Automatically generated information about the variable 'Provincia'

In the generated interactive report, we will be able to observe some previously extracted conclusions, such as the number of null values, the high correlation between certain variables, or the distribution of each variable.

4. CONCLUSIONS

Exploratory Data Analysis (EDA) constitutes a fundamental pillar in modern data science, providing a robust methodological framework for the initial understanding of complex datasets. As demonstrated throughout this guide, this process goes beyond simple preliminary data inspection, forming a critical phase that determines the quality and reliability of any subsequent analysis, whether it be rigorous scientific research or the development of advanced interactive visualizations.

The systematic application of EDA techniques, from verifying data integrity to analyzing multivariate correlations, allows for building a solid foundation for data-driven decision-making.

This guide has adopted a practical approach, using real air quality data that, although available in the national catalog datos.gob.es, are downloaded directly from their original source: the open data portal of Castilla y León. This dataset has allowed illustrating both the application of techniques and the specific challenges and considerations that arise when working with environmental data. The choice of Python as an analysis tool is due to its growing relevance in the data science ecosystem, providing an accessible yet powerful foundation for implementing these methods.

It is important to emphasize that the techniques presented constitute a basic starting point in data analysis. In more advanced applications, these methods can and should be complemented with more sophisticated techniques, such as advanced multivariate analysis, automated anomaly detection, or the use of machine learning techniques for exploring complex patterns.

We hope this guide serves as a practical resource for those starting in data analysis, providing a solid methodological foundation that can be applied and adapted to various datasets and specific contexts. See you soon!

5. NEXT STEPS

If you want to delve deeper into the fascinating world of exploratory data analysis, we suggest the following resources:

- Some freely available books that detail the process of exploratory data analysis and include test datasets and examples with code (R or Python) to illustrate the process:
 - [*Python for Data Analysis*](#): a fundamental book that extensively covers data analysis in Python, including EDA.
 - [*Exploratory Data Analysis with R*](#): a classic book focused on EDA using R.
 - [*R for Data Science*](#): an extensive resource on data science and exploratory analysis in R.
- In addition to books, the best way to learn data science is by practicing. Below, we provide links to tutorials and online courses with a significant amount of practical programming:
 - [*Comprehensive Data Exploration with Python*](#): a tutorial on Kaggle that guides you through a complete exploratory data analysis before training a machine learning model.
 - [*Exploratory Data Analysis with Python and Pandas*](#): a step-by-step tutorial on how to perform EDA using pandas.
 - [*Exploratory Data Analysis with Seaborn*](#): complete guide for visualization and EDA with seaborn.
- Lastly, here are some very useful additional resources that graphically compile the most relevant information:
 - [*Data Science Cheat Sheet*](#): DataCamp cheat sheets that summarize key data science concepts.
 - [*Seaborn Cheat Sheet*](#): Examples and cheat sheets for the Seaborn library, useful in EDA.
 - [*Pandas Cheat Sheet*](#): A practical summary of the most common operations with Pandas in Python.